# 让AI变成Super员工的秘密：高效训练Skills

原创 腾讯程序员 腾讯技术工程 2026年3月23日 02:36 广东 633人



作者：solonnliu

> 为什么 AI 很聪明，却总干不好复杂任务？不是模型不行，而是欠缺“业务 SOP”！本文复盘了打造 web-testing Skill 时的 4 个真实翻车教训，分享一套极具实操性的工程化训练心法（门禁规则、Checklist、自动迭代）。教你告别玄学 Prompt，把 AI 真正调教成能闭环交付的 S 级员工！不是 AI 不够强，而是大多数 AI 还没有被你训练成“懂业务、会自检、能稳定交付”的员工。

OpenClaw 火了以后，一个变化非常明显：几乎每个人都拥有了一个“什么都能干一点”的 AI 助手。

写文案、查资料、写代码、做分析、跑测试，它都能上。

但真正把 AI 拉进工作流之后，问题很快就暴露了：

**AI 不是不能干活，而是经常干得不稳、不深、不像一个真正懂你业务的人。**

它能执行，但不一定知道你这个场景里什么算“完成”。 它能输出，但不一定知道你团队真正要的交付物长什么样。 它能连续干很久，但上下文一长，它就会开始“失忆”、偷步骤、压缩细节，最后给你一个**“看起来做完了，实际上没做全”**的结果。

## 缘起

我最近一直在打磨一个 `web-testing` Skill，目标是：

给它任意一个网站的链接，它都能深入完整的探索到每一层页面的每一个细节，并输出完整的测试报告。测试报告包含总评分，站点地图，UI/UX 设计审查、功能测试、CURD全链路测试、每个测试点的截图等，自动梳理出每个页面的 bug 并按严重程度归类。输出格式包含可修改的 md 文件和更美观的 html 文件，人工审查没问题后，再交给 AI 按照报告自动修复。

**实现全自主的自动化测试、自动修复、再次自动测试验证，形成发现问题 → 解决问题的 AI 自驱动式全链路自动化测试，为产品质量保驾护航。**

当前还有很多细节需要打磨，但是主体功能已经完成。

这个过程给我最大的感受是：

**训练 Skill，本质上不是给 AI 多喂几句 Prompt，而是在把一个聪明但不稳定的通用模型，带成一个能独立扛活的业务员工。**

而且这件事，不玄学，甚至很工程化。

**先说结论：把 AI 训成 S 级员工，我现在最信这 4 件事**

1. **AI 的上下文一定有限。** 任务一复杂，它就会健忘。
2. **Skill 里不能只写“想要什么”，还要写“具体怎么做”。**
3. **光写清楚还不够。** 上下文一撑爆，AI 会自动忽略细节，所以一定要配可校验、可自测的 checklist 和门禁规则。
4. **Skill 不是一次写成的。** 最有效的方法永远是：跑一遍、复盘一次、让 AI 分析原因并自动改 Skill，再跑一遍，循环迭代。

如果只让我把这篇文章压缩成一句话，那就是：

> **Skill 的作用，不是让 AI 更聪明，而是让 AI 在你的业务里更可靠。**

## 一、为什么 AI 明明很强，还是当不了 S 级员工？

因为大模型默认拥有的是**通用能力**，不是**岗位能力**。

通用能力解决的是：

- 能不能理解任务
- 能不能大致做出来
- 能不能输出一个“像样的结果”

岗位能力解决的是：

- 你这个业务里什么才算真正完成
- 哪些步骤绝对不能跳
- 哪些细节最容易被漏掉
- 哪些错误过去已经真实发生过
- 最终交付物应该长什么样，谁来消费，怎么验收

这两者不是一个层级的问题。

很多人第一次用 AI，会被它的“会很多”震住；但真正把它扔进复杂任务里，你会发现它特别像一个非常聪明的新同学：

- 你这个业务里什么才算真正完成

- 哪些步骤绝对不能跳
- 哪些细节最容易被漏掉
- 哪些错误过去已经真实发生过
- 最终交付物应该长什么样，谁来消费，怎么验收

而 Skills 做的事，就是把这些**原本散落在你脑子里、团队经验里、事故教训里的内容，变成 AI 可以执行的岗位 SOP。**

换句话说：

**模型提供通用智力，Skill 提供业务操作系统。**

## 二、训练 Skill，为什么一定要先承认“AI 会失忆”？

这是我这次做 web-testing Skill 时感受最深的一点。

很多人训练 Skill，默认假设 AI 会一直按你说的做下去。但现实不是这样。

现实是：

- 任务一长，前面的约束会被稀释
- 阶段一多，后面的细节会被忽略
- 输出一复杂，AI 会主动压缩结构
- 上下文一接近极限，模型会优先保“看起来完成”，而不是保“严格完整”

所以，**复杂任务不是“写清楚一次”就结束了，而是必须被设计成“每一步都能自我校验”的流程。**

下面这张图，是我现在最认可的 **Skill 训练闭环**：




注意，图里最关键的不是“再次执行”，而是中间那一步：

**把错误写成规则，把经验写成 SOP，把 SOP 写成 AI 自己也能检查的门禁。**

## 三、web-testing Skill 是怎么一步步“训出来”的？

这部分我不讲抽象方法论，直接讲真实翻车。

因为真正有效的 Skill，几乎都不是靠想象写出来的，而是从一次次真实翻车里长出来的。

### 3.1 第一坑：AI 不是不会点页面，而是不知道“哪些地方必须点”

这是整个 `web-testing` Skill 演进里最关键的一次修正。

想要实现全站自动化测试，第一个重难点就是页面发现。如果页面都没发现完全，最终的结果肯定不是完整的。

然而我们要做的是具备通用性的 Skill，即随便拿过来一个网站都能深度分析到每一个页面，而不是针对某个具体网站的专属 Skill，没有固定的点击路径、公式做约束，只能让 AI 依靠我们的描述和规则自主发现。

举一个例子：一开始，AI 在页面发现阶段漏掉了一个关键页面：

```
/admin/product/deployment/{deploymentId}
```

这个页面并不在最显眼的位置，它藏在：

- 版本详情页的某个 Tab 里
- Tab 内容区的表格里
- 一个蓝色链接后面

听起来是不是很像真实业务系统里最常见的那种“藏得不深，但很容易被漏”的入口？

问题就出在这里。

AI 当时的策略是：

- 识别到了 Tab
- 知道有部署方案这个区域
- 也看到了蓝色链接
- 但没有把它当成“必须递归验证的新页面入口”

于是结果就变成了：

**它不是完全没探索，而是探索到一半，靠主观判断停下来了。**

大家可以看下面这张图，红框圈出来的就是被 AI 漏掉的部分。**深层页面入口，最容易被 AI“看见但没真正点进去”**

这类问题特别有代表性，因为它揭示了一个真相：

> **Web 测试的难点，很多时候不在“有没有能力点击”，而在“知不知道哪些交互必须被系统性穷举”。**

所以这次改 Skill，不是补一句“请更仔细”，而是直接把“仔细”拆成了可执行动作：

- 页面必须滚动到底再结束扫描

- Tab 切换后必须重新扫描链接
- 展开行里的链接必须实际点击验证
- 不能靠颜色或视觉主观判断链接是否重要
- 阶段结束前必须做递归终止自检

也就是说，Skill 里不能写：

> 请认真检查页面，不要遗漏入口。

而要写成：

> 当页面存在 Tab、展开行、子表格、蓝色链接时，必须执行 DOM 枚举与逐链接验证；只要还存在未验证链接，就禁止结束阶段 1。

这就是我现在很确定的一条原则：

**不要写原则，要写触发条件 + 必做动作 + 结束门槛。**

在进行修改完善后，发现的页面明显变多了，能下钻到隐藏更深的页面。下面第一张图是优化前，只识别到了2层页面。第二张图是优化后，识别到了4层页面，并标注了页面类型是只读还是可以 CURD 操作。

这个案例让我真正意识到：

**AI 不是天然会“穷举”的。穷举能力，很多时候要靠 Skill 强行教出来。**

### 3.2 第二坑：AI 会优先做“最像成果的那个”，然后漏掉其他交付物

后面我又踩到一个特别经典的坑。

阶段 4 的要求明明是输出三个文件：

- `sitemap.md`
- `test-report.md`
- `test-report.html`

结果 AI 只生成了 HTML。

为什么？

因为 HTML 最像“最终成果”。 它最显眼、最复杂，也最容易吸走模型的注意力和上下文预算。

于是 AI 的心理活动大概是这样的：

- 先把最复杂的做了

- 做着做着上下文变紧张了
- 后面两个相对“朴素”的产物就被自动忽略了

这件事很像很多新同学：

**不是故意偷懒，而是本能地优先做那个最有存在感的东西。**

所以这一轮修复，不是说“记得全部生成”，而是直接把顺序写死：

1. 先生成 `sitemap.md`
2. 再生成 `test-report.md`
3. 最后生成 `test-report.html`
4. 每生成一个文件，立刻检查是否存在且大小大于 0
5. 阶段结束做阻断式完整性检查，不通过就不能宣布完成

这是 Skill 设计里特别容易被忽视的一点：

**复杂任务里，顺序本身就是质量控制。**

### 3.3 第三坑：不要假设 AI 懂工程约束，它经常会选“看起来最省事”的方案

还有一次，AI 为了生成带截图的 HTML 报告，走了一条“看起来很聪明”的捷径：

- 用 `python3 -c` 直接拼长脚本
- 把截图数据做成 base64
- 一起塞进命令行

结果很直接：

**Shell 命令长度上限被打爆，整条命令失败。**

这次教训特别大，因为它说明了一件事：

> **模型并不天然具备你当前执行环境里的工程常识。**

它可能“知道有这种写法”。 但它不知道这在你的环境里是否稳定、是否可维护、是否会炸。

所以后面我把一类以前觉得“没必要写这么细”的规则，也明确写进了 Skill：

- 禁止用 `python3 -c` 生成长报告
- 禁止用超长 `echo "..." > file` 硬写大文件
- 优先直接写文件，或者先写脚本文件再执行
- HTML 报告禁止 base64 内嵌截图，统一改成本地路径引用

从这以后，我对 Skill 又多了一层理解：

**Skill 不只是业务手册，它还得是 AI 的“工程生存指南”。**

否则逻辑明明全对，最后死在命令行里。

### 3.4 第四坑：最危险的不是“没生成”，而是“看起来生成了”

这是最近这轮迭代里，我觉得最值钱的一次翻车。

本来以为大成了，但仔细一检查发现个很扎心的问题：

**为什么以前每个页面都有详细测试报告，现在最新版没有了？**

表面看，报告其实是有的。 甚至还有问题列表、CRUD 结果、结论。

但一对模板才发现：

真正关键的“逐页面详细模块”被悄悄压缩掉了。

少了什么？

- 每个页面独立模块
- UI/UX 审查
- 功能测试明细
- 本页问题汇总
- HTML 的 page-card 结构

根因也很现实：

**前面测试过程已经消耗了大量上下文，到了最后生成报告时，AI 自动进入“节约模式”，开始压缩结构。**

这类错误为什么最危险？

因为它不是“完全没有”，而是“看上去像有”。

它最容易骗过第一次验收。

所以这一次修的核心，不是“补内容”，而是补**结构门禁**。

**复杂任务如果没有门禁，AI 很容易从“完整交付”滑向“看起来完成”**



我后来专门在 Skill 里补了一套类似这样的检查：

```
报告结构完整性 checklist（示意）
- [ ] 页面模块数 = sitemap 页面数
```

```
- [ ] 每页都有截图
- [ ] 每页都有 UI/UX 审查
- [ ] 每页都有功能测试结果表
- [ ] 每页都有本页问题汇总
- [ ] HTML 中 page-card 数量 = 页面数
- [ ] 问题汇总表存在
- [ ] 修复计划表存在
- [ ] 数据清理记录存在
```

关键不是这个 checklist 写得多漂亮。 关键是它要变成**门禁**:

> **上一项没过，下一项不能开始；最终结构没过，整个阶段不能宣告完成。**

这就是为什么我现在越来越不相信“靠 AI 自觉”了。

复杂任务里，**自觉是加分项，门禁才是底线保障。**

## 四、从这个案例里，我总结出一套真正能用的 Skill 训练方法

如果你也想把一个通用 AI 训练成你业务里的“靠谱员工”，我现在建议直接按下面这套顺序来。

### 方法 1：先让 AI 在真实任务里跑起来，再谈训练

别一上来就闭门写一大坨 Skill。

先让 AI 真做 3～5 次真实任务。

重点不是看它做得多惊艳，而是看它怎么错:

- 漏了什么
- 跳了哪一步
- 哪些地方看起来做了，其实没做
- 哪些错误会重复出现

**没有真实翻车，就很难写出真的有用的 Skill。**

### 方法 2：Skill 里不止要写“做什么”，还要写“怎么做”

这是最容易被忽略的一点。

坏写法:

- 请仔细检查页面
- 请保证报告完整
- 请在结束前自查

好写法：

- 当页面存在 Tab/展开行/子表格时，切换后必须重新枚举链接
- 每生成一个交付物，立刻验证文件存在且大小大于 0
- 页面模块数必须等于站点地图页面数，否则报告不合格

区别在哪？

前者是在提要求。 后者是在写 SOP。

而 Skill 要的，恰恰是 SOP。

### 方法 3：只写清楚还不够，一定要给 AI 配 checklist 和门禁

很多人会以为：

“我已经描述得很清楚了，AI 应该不会漏吧？”

很遗憾，复杂任务里真不是这样。

因为上下文一长，AI 会自动做几件事：

- 压缩不那么显眼的步骤
- 忽略长尾细节
- 优先保住“看起来像成果”的部分
- 把“差不多完成”当成“已经完成”

所以，Skill 里必须有两层东西：

1. **checklist**：告诉 AI 要检查什么
2. **门禁**：告诉 AI 没检查过就不能往下走

没有 checklist，AI 不知道该怎么自检。 没有门禁，AI 就算知道也可能跳过去。

这一步，特别像带新人：

你不能只告诉他“注意质量”。 你得告诉他：

- 先做什么
- 做到什么算完成
- 怎么验收
- 不通过怎么办

### 方法 4：效果不好时，不要只自己改，要让 AI 参与复盘并自动改 Skill

这是我这次最推荐大家立刻就能用起来的一招。

很多人调 Skill 的方式是：

- 跑一遍
- 觉得不满意
- 自己手改 Prompt
- 再跑一遍

这样当然也行，但效率并不高。

更好的方式是：

1. 让 AI 跑完一遍真实任务
2. 把不理想的地方明确指出来
3. 让 AI 分析：错在哪里、为什么错、根因是什么
4. 让 AI 直接提出应该如何修改 Skill
5. 让 AI 自动完善 Skill
6. 再跑一遍验证
7. 重复这个闭环，直到效果比较满意

也就是说，**AI 不只是执行者，还应该成为 Skill 的共同调参者。**

我现在很喜欢用一类这样的迭代指令：

```
请基于本次执行结果，对当前 Skill 做一次复盘：
1. 哪些输出没有达到预期？
2. 这些问题分别属于：页面发现、交付完整性、工程约束、结构完整性、消费场景适配中的哪一类？
3. 根因是什么？是规则缺失、规则不明确、没有门禁，还是上下文过长导致细节被忽略？
4. 请给出应补充到 Skill 中的具体规则，要求包含：触发条件、必做动作、自检方式、不通过后果。
5. 直接输出修改后的 Skill 片段，并说明这次修改预期解决什么问题。
```

这个方法特别适合复杂业务场景，因为它能把“这次错了”变成“以后尽量别再错”。

Skill 一旦进入这个循环，就会越来越像一个真正会吸收经验的业务系统，而不只是一个长 Prompt。

**web-testing 的文件目录**

下面是 web-testing 的目录

```
web-testing/
├── SKILL.md
├── references/
|   ├── checklist-template.md
|   ├── report-template.md
|   └── ui-ux-checklist.md
```

这个 `web-testing` Skill 的目录里，**真正构成能力核心的文件主要有 4 个**：

- `SKILL.md`：主控文件，定义触发条件、执行流程、门禁规则、失败模式
- `references/checklist-template.md`：执行过程的“进度控制器”和“阶段门禁表”
- `references/report-template.md`：最终 Markdown / HTML 报告的输出契约
- `references/ui-ux-checklist.md`：UI/UX 审查时的详细评分参考标准

通过主 `SKILL.md` + `checklist` 的形式，确保了 AI 在执行过程中不健忘，按照期望交付成果。

## 五、训练 Skill，不是在抬高上限，而是在抬高下限

很多人提到 AI，总会讨论上限：

- 这个模型够不够聪明
- 那个模型推理强不强
- 它会不会写代码、做分析、出报告

但真到了业务里，决定体验的往往不是上限，而是下限。

你最怕的不是 AI 偶尔不够惊艳，最怕的是它：

- 这次做得很好，下次突然漏半页
- 这次三份文件都有，下次只交一份
- 这次结构完整，下次悄悄缩水
- 这次图片能看，下次链接全挂

所以 Skill 真正解决的是：

**让 AI 的交付质量从“靠状态”变成“靠机制”。**

这也是为什么我现在越来越觉得：

> **训练 Skill，本质上不是增强 AI 的天赋，而是在建立 AI 的职业素养。**

它要知道流程，它要记住教训，它要会自检，它要过门禁，它要稳定交付。

这才是一个“S 级员工”的样子。

## 六、结语

做完这个 `web-testing` Skill 之后，我最大的感受就是：

**要把自己的业务经验、失败教训、质量标准、交付契约，沉淀成 AI 可以反复执行的 Skill。**

**训练 Skill，不是在教 AI 做题，而是在带一个聪明新人上岗。**

你要让它先去干活，你要允许它犯错， 你要把错复盘出来，你要把经验写回 Skill， 然后你再让它继续干。

反复几轮之后，它就会慢慢从“能干活”，进化成“能把活干好”。

而这，才是 AI 真正变成 S 级员工的开始。




腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号




阅读 2.1万

留言 1

写留言



huan666 北京 4月1日

1

👍👍👍